Chauncey Thompson

Intro &Hypotheses

The goal of this project was to investigate how SIMD (single instruction, multiple data) instructions affect the performance of numerical programs. SIMD allows a processor to apply one instruction to multiple values simultaneously, likely completing calculations much faster than scalar execution would.

For this project I chose these three kernels:

- SAXPY: y[i] = a*x[i] + y[i]
- Elementwise multiplication: z[i] = x[i] * y[i]
- Dot product: sum += x[i] * y[i]

Each kernel was tested using scalar, default auto-vectorized, and CPU-specific optimized compilation. I hypothesize that SIMD will provide greater speedup for all kernels when arrays are small. When the arrays are large, memory access will become more important.

Hardware and Software Environment

All experiments were performed on a Windows computer with the following hardware:

Component	**Specification**
Processor	Intel Core i5-14400F
Efficiency cores	4
Total CPU threads	16
Memory	32 GB DDR5-4800
Memory configuration	Two 16 GB modules
L3 cache	20 MB
Operating system	Windows 11
Compiler	GCC 16.2.0

My i5-14400F supports AVX2 instructions, allowing 256-bit vector operations. Those vectors can hold eight Float32 values or four Float64 values. The benchmarks used one software thread rather than parallelizing work across multiple CPU cores.

Three compiler configurations were used:
Scalar - Disables automatic loop and basic-block vectorization
Default - Enables GCC optimization and automatic vectorization
Native - Enables optimizations targeted at the current CPU

The scalar configuration used "-O3" with "-fno-tree-vectorize" and "-fno-tree-slp-vectorize".
The default configuration used "-O3"
The native configuration used "-O3 -march=native".
The scalar build served as the baseline for calculating speedup of vectorization. The default and native builds demonstrated the effects of automatic and CPU-specific optimization.

Timing procedure

Execution time was measured by using a high-resolution Windows timer based on "QueryPerformanceCounter". The benchmarks tested multiple array sizes. The processed results use the median execution times rather than relying on a single measurement. The raw measurements were kept in CSV files. For SAXPY and dot product, two floating-point operations were counted per element. For multiplication, one floating-point operation was counted per element.

Performance was analyzed in three different ways:

- **Speedup** = Scalar execution time / Optimized execution time
- **GFLOP/s** = Total floating-point operations / Execution time in seconds / 10^9
- **Effective bandwidth** = Logical bytes accessed / Execution time

Correctness and reproducibility

I had the experimental programs include correctness checks to make sure that the optimization did not produce unacceptable output differences. Correctness is specifically important for dot-product reductions because changes to floating-point addition order can cause small numerical differences.

ChatGPT decided the best way to go about it was to have the benchmark use input values of 1.0 and 2.0, allowing expected outputs to be analytically calculated. SAXPY was checked to ensure that paired positive and negative updates restored the original array values, while elementwise multiplication was checked for exact output values of 2.0. The dot-product results were compared against the expected value of (2N), using an absolute error tolerance of $(10^{-5}(2N) + 10^{-6})$. The same dot-product tolerance was used for Float32 and Float64. All recorded benchmark trials passed the correctness checks and can be found in the bench__.csv files under "raw".

Chauncey Thompson

Scalar vs. SIMD Performance

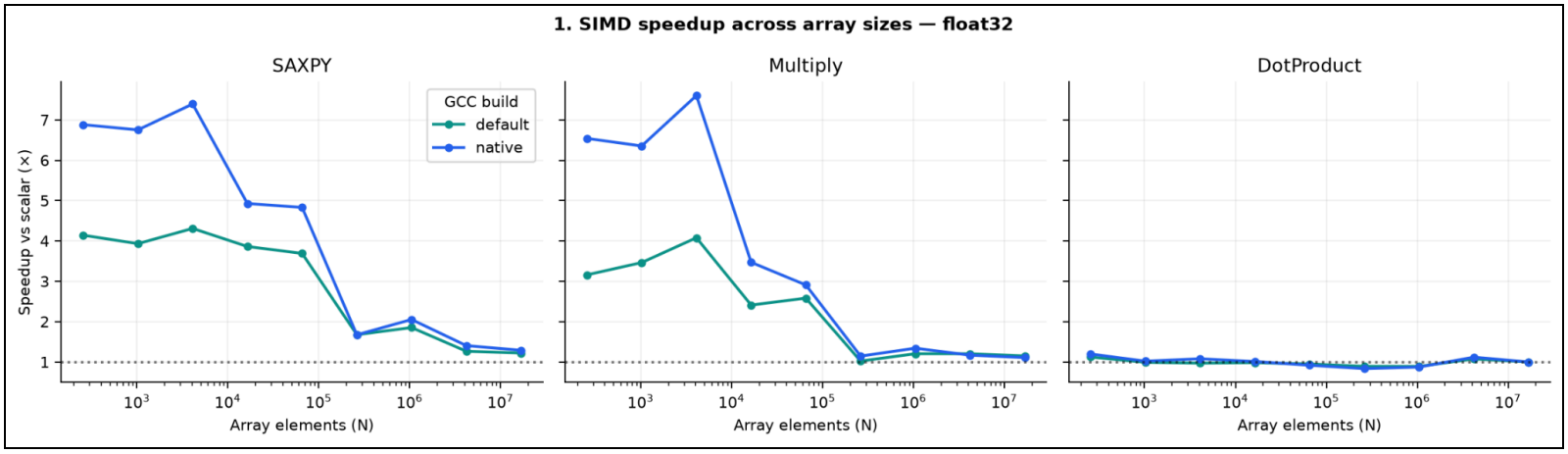


Figure 1: SIMD speedup

Figure 1 compares default and native SIMD speedups against scalar execution for the three kernels across different array sizes. For 16,384 Float32 elements, the native implementation achieved approximately 4.93x SAXPY, 3.47x Elementwise multiplication, and 1.01x Dot product. These results show that SAXPY and multiplication benefit substantially from SIMD when the arrays are relatively small, but the speedup decreases at larger array sizes. That suggests that SIMD benefits diminish when data movement makes up most of the execution time. Dot product not seeing much speedup goes against my hypothesis.

Computational throughput

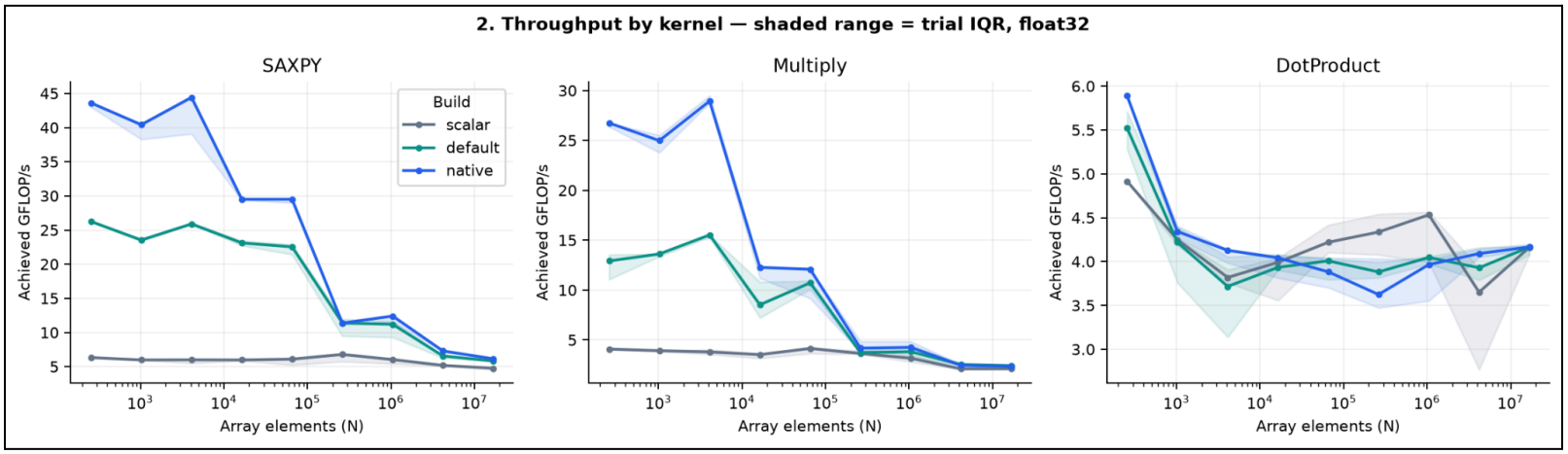


Figure 2: Throughput by kernel

Figure 2 shows computational throughput in GFLOP/s for the three compiler configurations. For multiplication with 10^6 elements, the native execution time was approximately 0.24615 milliseconds. Multiplication performs one FLOP per element, so the achieved throughput was: GFLOP/s = 1,048,576 / (0.00024615 × 10^9) → Achieved throughput ≈ 4.26 GFLOP/s

Native achieved higher throughput than scalar execution for SAXPY and multiplication, especially with smaller arrays. For the larger working sets, the throughput advantage diminished. This shows that vector calculations alone do not determine overall performance. Although the compiler generated vectorized loop versions for the dot product, it did not get the large speedups seen with the other two kernels.

Memory Locality and Cache Effects

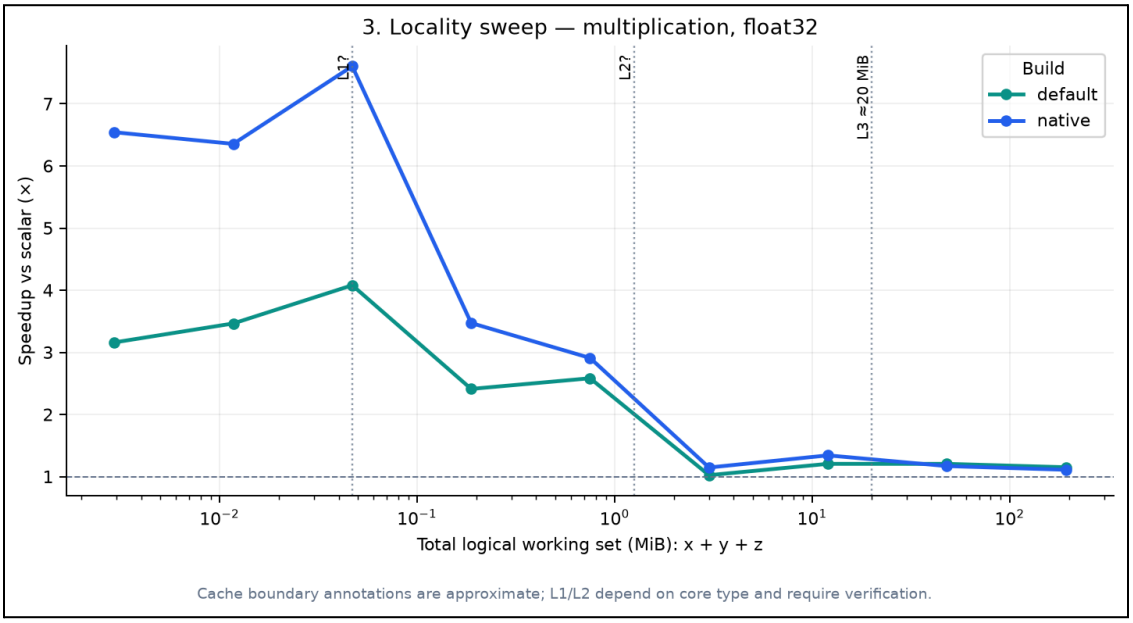


Figure 3: Locality sweep

Figure 3 looks at the multiplication speedup as the working-set size increases. Elementwise multiplication uses three arrays. With Float32, each element occupies four bytes. Therefore, the total array working set is approximately: $3 \times N \times 4$ bytes. For 10^6 elements, this would be about 12 MiB of array storage. The small working sets benefitted from fast CPU caches, while larger working sets required more data to travel through the main memory. The measured speedups generally decreased as the working set became larger which is consistent with the other two graphs.

Chauncey Thompson

Memory alignment

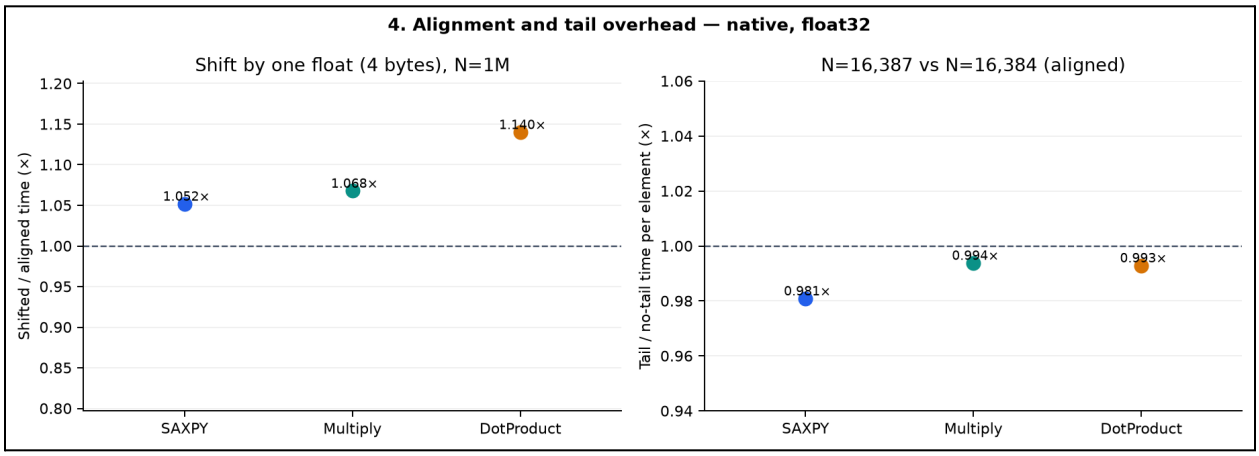


Figure 4: Alignment and tail overhead

The alignment experiment (left side plot) compared arrays beginning on a 32-byte boundary against arrays shifted forward by four bytes. The alignment penalty was calculated as: Alignment ratio = Unaligned time / Aligned time. A ratio greater than 1 shows that the unaligned array took longer. For an array size of 10^6 elements, the native Float32 results showed mild unaligned penalties in the cases plotted, about 6% for SAXPY, 8% for multiplication, and 14% for dot product. This indicates that alignment can have an effect on performance, but the penalty was not always large.

The tail experiment (right side plot) compared array lengths divisible by the vector width against lengths containing three additional elements. For example, 16,384 elements divide evenly into groups of eight Float32 values, while 16,387 elements leave three extra values. The experiments compared normalized time per element to avoid confusing the additional work with tail-processing overhead. The results showed little consistent performance penalty for three tail elements. This makes sense because three extra elements represent only a small part of the total work for the tested arrays.

Chauncey Thompson

Strided memory access

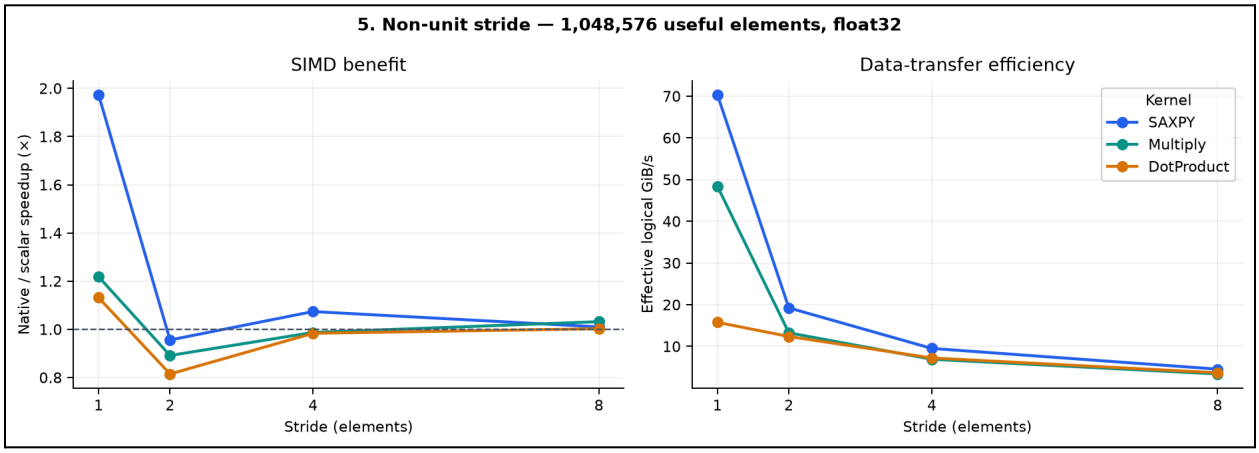


Figure 5: Stride

The stride experiment tested memory access distances of 1, 2, 4, and 8 elements. A stride of one processes consecutive values, while a stride of eight accesses elements spaced eight positions apart. The number of useful calculations was kept consistent while the input arrays became more spread out in memory. The results showed that native SIMD was particularly effective for unit-stride operations. At larger strides, speedup generally went down toward 1.0x. This can happen because larger strides use cache lines less efficiently and make vector memory access more complicated. The measured effective bandwidth also decreased with increasing stride in the plotted cases. Overall, this experiment demonstrates that memory access patterns are essential to SIMD performance, even when the mathematical operations are unchanged.

Float32 vs. Float64

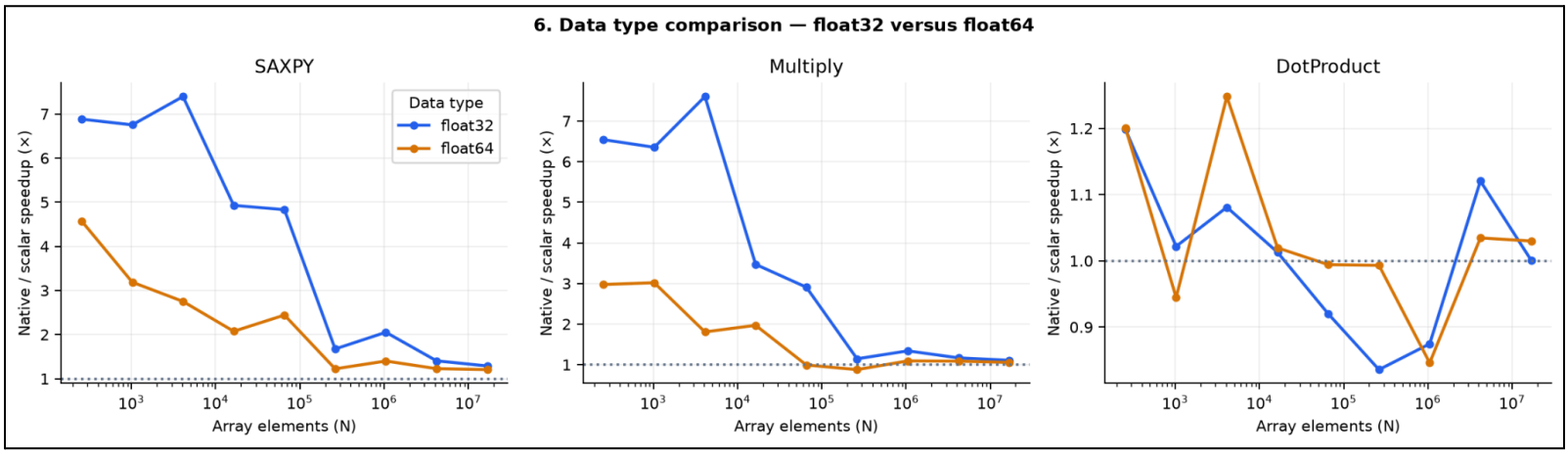


Figure 6: Float32 vs Float64

The data-type experiment compared SIMD speedup using single and double precision floating-point values. Float32 uses four bytes, while Float64 uses eight bytes. A 256-bit SIMD vector can hold eight Float32 values but only four Float64 values. The measurements showed greater SIMD speedup for Float32 in SAXPY and multiplication, particularly for smaller arrays.

This lines up with Float32's reduced memory requirements and larger number of elements per SIMD vector. However, dot product didn't show much SIMD speedup in both data types.

Verification of SIMD Vectorization

Compiler vectorization reports were generated to verify that GCC created SIMD loop implementations. The reports showed the following results:

Configuration	Float32 vector widths	Float64 vector widths
Scalar	Automatic vectorization disabled	Automatic vectorization disabled
Default	8 and/or 16 bytes	16 bytes
Native	16 and/or 32 bytes	16 and/or 32 bytes

GCC reported vectorization for all three kernel functions in the default and native builds. The native configurations included 32 byte vector operations, consistent with AVX2 support on the processor. One important limitation is that compiler-generated loop versions do not necessarily prove that every runtime case used a vectorized branch. In particular, runtime-variable strides may use different optimized and nonoptimized paths. Therefore, the compiler reports provide evidence of vectorization capability, while the measured execution times show the actual performance behavior. The full compiler reports are in the GitHub under “reports”.

Roofline Performance Analysis

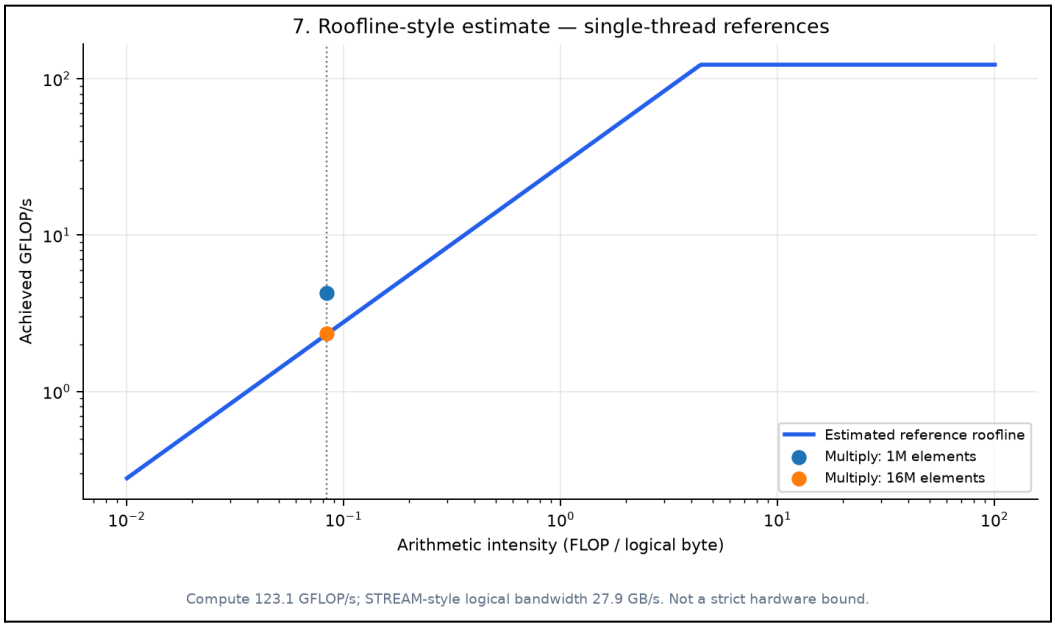


Figure 7: Roofline Performance

A roofline-style model was used to look at whether the multiplication kernel was more likely limited by floating-point computational throughput or memory bandwidth. The model uses the relationship: Attainable performance = min(Compute throughput, Bandwidth x Arithmetic intensity)

Two additional measurements were collected:

Measurement	**Result**
Single-thread AVX2/FMA throughput reference	123.08 GFLOP/s
Highest measured STREAM-style effective bandwidth	27.88 GB/s

Arithmetic intensity represents the number of floating-point operations performed per byte of logical memory traffic. Elementwise multiplication performs one FLOP per element and has approximately 12 logical bytes of traffic: two Float32 reads and one Float32 write.

Its arithmetic intensity is: AI = 1 / 12 ≈ 0.0833 FLOP/byte
Using the measured bandwidth reference: 27.88 × 0.0833 ≈ 2.32 GFLOP/s

For the largest tested Float32 multiplication array, the native implementation hit approximately 2.35 GFLOP/s. This is close to the estimated memory-bandwidth reference, supporting the idea that large-array multiplication is limited mainly by data movement rather than peak floating-point computational ability. For smaller arrays, measured performance can go past this DRAM-based reference because data sitting in cache may be supplied at a higher effective rate. The model uses logical data traffic, a separately measured memory-bandwidth benchmark, and a synthetic FMA compute-throughput benchmark rather than exact hardware performance counters.

Negative Results and Experimental Limitations

One important result was that SIMD did not significantly speed up every kernel. Dot product received little speedup in many cases despite GCC reporting vectorization. For example, with 10^6 Float32 elements, the native version had a 0.875x speedup, meaning it was slower than scalar execution. This result may show accumulation dependencies, instruction scheduling, or other limitations of the generated reduction code. Compiler vectorization alone does not guarantee a reduction in execution time. The stride experiment also showed that nonconsecutive accesses could reduce SIMD benefits. These are useful negative examples because they demonstrate that SIMD effectiveness depends on the complete execution workload, not just on the presence of vector instructions.

Chauncey Thompson

Conclusion

This project showed that SIMD can significantly improve the performance of kernels, but its benefits depend on the type of computation and memory-access behavior. SAXPY and elementwise multiplication saw large speedups with smaller arrays. Their performance advantage decreased as working sets got bigger, consistent with increasing memory limitations. Memory alignment had a modest effect, while short tail lengths produced little measurable overhead. Strided accesses reduced SIMD effectiveness by substantially, demonstrating the importance of consecutive memory access.

AI-Use Disclosure

ChatGPT was used to assist with developing and organizing the benchmarking programs, explaining SIMD concepts and compiler flags, generating plotting and analysis scripts, interpreting measurements, and preparing the report structure.